FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Enhancing Centralized Multi-Robot Path Planning and Coordination

Afonso Tomás de Magalhães Mateus

Mestrado em Engenharia Eletrotécnica e de Computadores

Supervisor: Prof. Pedro Luís Cerqueira Gomes da Costa

Second Supervisor: Prof. Diogo Miguel Rodrigues Matos

May 8, 2026

Abstract

This dissertation addresses the problem of centralized coordination of multiple autonomous mobile robots operating in shared environments. As robotic fleets become increasingly prevalent in industrial, logistics, and service applications, the ability to compute coordinated, collision-free trajectories under real-world constraints becomes a critical requirement. This challenge is commonly formalized as the Multi-Agent Path Finding (MAPF) problem, which seeks to determine feasible paths for multiple agents from given start locations to specified goal locations while avoiding conflicts over time.

MAPF is computationally hard in its general form, which has motivated the development of practical planning approaches that trade optimality for scalability and real-time performance. In centralized fleet coordination systems, priority-based planners are often adopted due to their predictable behavior and computational efficiency. In particular, the centralized coordination framework developed at the Institute for Systems and Computer Engineering, Technology and Science (INESC TEC), within the Centre for Robotics in Industry and Intelligent Systems (CRIIS) and the Industry and Innovation Laboratory (iiLab), relies on a sequential Time-Enhanced A* (TEA*) planner, which incorporates temporal reasoning and space–time reservations to ensure conflict-free execution. While well suited for real-world deployment, this sequential structure introduces a significant limitation: planning outcomes are highly sensitive to the chosen robot priority ordering, and exhaustive exploration of all orderings is infeasible due to factorial growth with fleet size.

The main motivation of this work is to reduce the sensitivity of sequential TEA*-based planning to robot ordering without compromising the existing centralized architecture. Rather than replacing the underlying planner, this dissertation explores the use of parallel computation to improve solution robustness within a fixed planning time budget. The proposed approach executes multiple TEA* instances concurrently, each operating on a different priority ordering of the same MAPF instance, and selects the most suitable solution among those evaluated.

The solution builds upon a centralized planning framework in which a Supervisor issues planning requests composed of a set of agents and their associated start and goal vertices. Within the Path Planner, heuristic functions generate diverse priority orderings that are processed independently in parallel threads. Although TEA* remains sequential within each thread, the overall planning process benefits from parallelism at the system level. Alternative execution strategies are also discussed, including early solution selection based on partial thread completion or fixed time budgets, enabling a tunable trade-off between planning latency and exploration depth.

Finally, the document outlines a structured work plan guiding the implementation and experimental evaluation of the proposed solution. Overall, this dissertation aims to contribute a practical and extensible approach for improving centralized multi-robot coordination by leveraging parallel planning to enhance robustness and solution quality in TEA*-based fleet coordination systems.

Contents

List of Figures

List of Acronyms

APF	*Artificial Potential Field*
CCBS	*Continuous-Time Conflict-Based Search*
CBS	*Conflict-Based Search*
CBSH	*Heuristic-Guided Conflict-Based Search*
CBSH-RTC	*Real-Time Conflict-Based Search with Heuristics*
CRIIS	*Centre for Robotics in Industry and Intelligent Systems*
ECBS	*Enhanced Conflict-Based Search*
INESC TEC	*Institute for Systems and Computer Engineering, Technology and Science*
MAPF	*Multi-Agent Path Finding*
NP-hard	*Nondeterministic Polynomial-time Hard*
ROS	*Robot Operating System*
SIPP	*Safe Interval Path Planning*
TEA*	*Time-Enhanced A-Star*
iiLab	*Industry and Innovation Laboratory*

Chapter 1

Introduction

1.1 Context

The increasing adoption of autonomous mobile robots in industrial, logistics, and service environments has intensified the need for effective coordination of multiple agents operating in shared spaces. In such scenarios, robots must navigate efficiently while avoiding collisions and mutual interference, often under strict temporal and operational constraints. As fleet sizes grow, coordination becomes a key challenge that directly impacts system performance, scalability, and reliability.

This problem is commonly formalized as the MAPF problem, which focuses on computing collision-free trajectories for multiple agents moving from initial to goal states in a shared environment [1]. By explicitly modeling agent interactions over time, MAPF has become a central abstraction for multi-robot coordination in both centralized and decentralized planning frameworks.

MAPF is computationally challenging, with optimal variants being NP-hard (Nondeterministic Polynomial-time hard) in the general case [2]. Consequently, a wide spectrum of solution approaches has been proposed, ranging from optimal but computationally demanding methods to heuristic and approximate techniques that favor scalability and real-time performance [3]. Within this landscape, MAPF serves as a foundational framework for studying coordination, conflict resolution, and scalability in multi-robot systems.

1.2 Motivation and Problem

This dissertation is developed in the context of research on centralized multi-robot fleet coordination conducted at INESC TEC, namely at CRIIS and the iiLab. Within this context, a centralized fleet coordination framework has been proposed to address multi-robot navigation under realistic operational constraints, relying on a shared graph-based environment representation and a sequential TEA* planning component [4]–[6].

In this framework, robots are planned sequentially according to a predefined priority order, which ensures conflict avoidance by construction but makes the resulting solution highly sensitive

to robot ordering. Different priority permutations may lead to substantially different outcomes, particularly in constrained environments with shared resources. As the number of possible orderings grows factorially with fleet size, exhaustive exploration becomes computationally infeasible.

These limitations motivate the investigation of parallel planning strategies. By exploiting multithreading to evaluate multiple priority orderings concurrently, the impact of ordering sensitivity can be reduced while preserving compatibility with the existing TEA*-based coordination framework.

1.3 Objectives

The main objective of this dissertation is to improve the robustness and effectiveness of a centralized TEA*-based multi-robot planner by reducing its sensitivity to robot prioritization through parallel planning. To this end, the impact of robot ordering on the feasibility and performance of sequential TEA*-based planning is analyzed, and a multithreaded approach is designed in which multiple TEA* instances are executed in parallel, each considering a different priority ordering. Heuristic strategies are developed to systematically generate and distribute priority orderings across parallel threads, and the proposed approach is integrated into an existing centralized fleet coordination framework. Finally, the solution is experimentally evaluated in representative multi-robot scenarios, assessing solution quality and computational performance.

1.4 Potential Contributions

This dissertation builds upon the centralized fleet coordination system described by Matos et al. [4] and proposes a multithreaded extension of a sequential TEA*-based planner that enables the parallel exploration of multiple robot priority orderings. The primary expected contribution is the ability to obtain higher-quality coordination solutions within the same planning time, without modifying the core architecture of the existing system.

By evaluating a broader set of priority orderings, the proposed approach aims to improve robustness and performance in constrained multi-robot scenarios. Additionally, this work provides empirical insights into the effects of parallelism in priority-based planning, including comparative evaluation against alternative approaches, contributing to the design of future centralized fleet coordination systems.

1.5 Document Structure

This document is organized into five chapters. Chapter 1 introduces the problem and objectives. Chapter 2 presents the background and related work. Chapter 3 describes algorithmic approaches to MAPF and the baseline fleet coordination framework. Chapter 4 presents the preliminary solution and work plan. Finally, Chapter 8 concludes the document and outlines its key conclusions.

Chapter 2

Background and Fundamental Aspects

This chapter presents the fundamental concepts and background knowledge required to contextualize the research developed in this dissertation. Section 2.1 and Section 2.2 introduces a formal definition of the MAPF problem and its representation, respectively. Section 2.3 explore classical graph search algorithms that serve as building blocks for many MAPF methods. Finally, Section 2.4 introduces foundational notions of parallel and concurrent computing.

2.1 MAPF Problem: Formal Definition

The MAPF problem concerns the computation of collision-free trajectories for a group of cooperative agents that must move from their initial states to their target states while operating in a shared environment. Each agent must reach its goal without colliding with other agents, while a global performance criterion is optimized. This general problem formulation follows the, widely adopted, conceptual definition of MAPF presented by Stern et al. [1]. While that work instantiates the problem on graph-based environments, the underlying definition is independent of the specific representation and can be abstracted to more general state spaces.

From a formal perspective, let there be a set of m agents $A = \{a_1, a_2, \ldots, a_m\}$. Each agent $a_i \in A$ is associated with an initial state s_i and a goal state g_i.

A solution for an agent a_i is defined as a trajectory P_i, which specifies the evolution of the agent's state over time. In discrete formulations, a trajectory can be represented as a sequence of states

$$P_i = \langle p_i^0, p_i^1, \ldots, p_i^T \rangle \tag{2.1}$$

where $p_i^0 = s_i$ and $p_i^T = g_i$. The explicit consideration of time is essential in MAPF, as interactions and conflicts between agents arise from the simultaneous execution of their trajectories in a shared space-time domain [1].

A valid MAPF solution must ensure that no two agents are in conflict during their motion, where conflicts arise when agents attempt to occupy incompatible states in space and time. The most common type of conflict is the position conflict, which occurs when two agents occupy the same state at the same time step, i.e., $p_i^t = p_j^t$ for $i \neq j$. Other types of conflicts may arise depending on

the motion model and representation adopted, but all correspond to violations of mutual feasibility constraints among agents [1].

The objective of MAPF is to compute a set of conflict-free trajectories $\{P_1, P_2, \ldots, P_m\}$ that minimize a global cost function C. This cost function is defined over the joint set of agent trajectories and encodes the chosen optimization criterion. Common objectives include minimizing the time at which the last agent reaches its goal (makespan) or minimizing the sum of individual agents' traversal costs, as commonly considered in the literature. Formally:

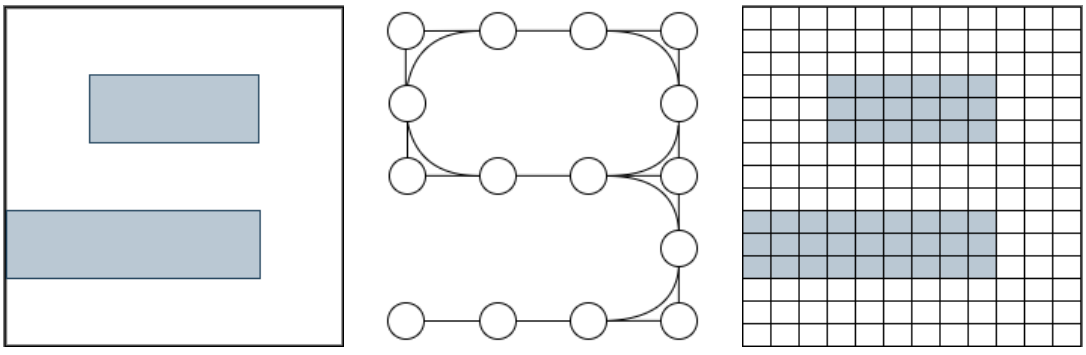
Find $\{P_1, P_2, \ldots, P_m\}$ **such that** all trajectories are conflict-free and $C(\{P_i\})$ is minimized.

The MAPF problem is combinatorial in nature and is NP-hard in the general case, as formally established in earlier works [2], [7] and confirmed in recent complexity analyses [8], [9]. This inherent complexity motivates the development of planning algorithms that aim to balance solution quality with computational efficiency.

2.2 MAPF Problem: Representation

Although the MAPF problem can be defined independently of the underlying environment representation, practical solutions require instantiating the problem on a concrete state space. Different environment representations have been adopted in the MAPF literature, primarily differing in how space and time are modeled and in the resulting level of abstraction.

Common approaches include discrete representations, such as grids and graphs, where agents occupy a finite set of states and move between neighboring states in discrete time steps, as well as formulations based on continuous space and time, where agent motion is described by continuous trajectories and conflicts arise from geometric interactions. Figure 2.1 illustrates how the same workspace can be modeled under different discrete representations.



(a) Workspace layout (b) Graph-based representation (c) Grid-based representation

Figure 2.1: Illustration of different discrete environment representations for the same workspace.

While continuous formulations offer higher modeling fidelity, discrete representations are more commonly adopted in classical MAPF settings due to their simplicity and structured abstraction. In the remainder of this section, the focus is placed on grid and graph-based representations, which provide a flexible and well-defined framework for modeling structured environments.

2.2.1 Grid-Based Representation

In grid-based MAPF formulations, the environment is discretized into a regular two or three-dimensional lattice of uniformly sized cells, each classified as free or occupied. Agents move between neighboring cells according to a fixed connectivity structure, yielding a simple and intuitive abstraction that has been widely adopted in early MAPF and multi-robot navigation studies [1], [10].

A key advantage of grid-based representations is their simplicity and direct correspondence to occupancy grids used in robotic perception. However, grid resolution introduces an inherent trade-off: coarse grids may fail to capture narrow passages or thin obstacles, while fine grids incur substantial memory and computational costs [10], [11]. In addition, the discrete nature of grid-based motion complicates the integration of continuous-time dynamics, often requiring post-processing to obtain executable trajectories [11].

For these reasons, many contemporary MAPF approaches favor graph-based representations, which generalize grids by allowing arbitrary topologies and connectivity patterns, making them a convenient abstraction for modeling structured environments in MAPF [1].

2.2.2 Graph-Based Representation

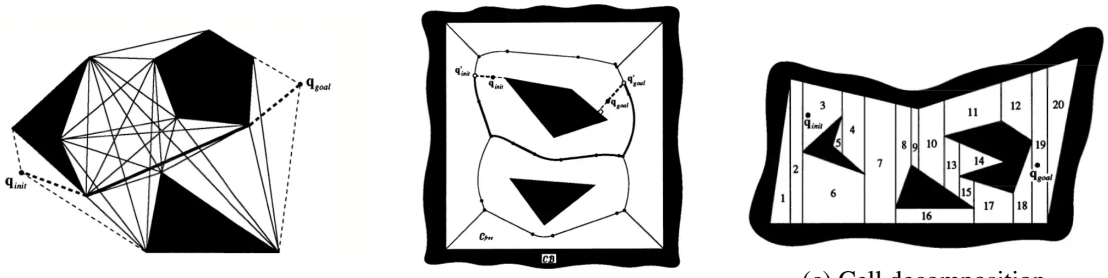
In graph-based formulations of the MAPF problem, the environment is modeled as a discrete graph that captures admissible agent states and their connectivity, following the modeling approach described by Stern et al. [1].

Formally, the environment is represented as a graph $G = (V, E)$, where each vertex $v \in V$ corresponds to a valid state that an agent may occupy, and each edge $(u, v) \in E$ encodes a feasible transition between states. The graph may be directed or undirected, depending on the characteristics of the environment. Under this representation, agent trajectories evolve in discrete time steps and are constrained to lie on the graph, such that each state p_i^t corresponds to a vertex in V and successive states satisfy $p_i^{t+1} = p_i^t \quad \text{or} \quad (p_i^t, p_i^{t+1}) \in E$ as commonly assumed in graph-based multi-robot and MAPF formulations [2].

2.2.2.1 Graph Construction Approaches

Several classical approaches have been proposed to construct graph-based representations of planning environments. These include roadmap-based approaches such as visibility graphs, which connect mutually visible obstacle vertices [12], Voronoi diagram-based representations that emphasize obstacle clearance [13], and cell decomposition techniques that subdivide the free space into regions whose adjacency defines a connectivity graph [14], [15]. Representative examples of

these constructions are illustrated in Fig. 2.2. However, in this dissertation, the focus is on MAPF formulations in which the underlying graph is assumed to be given, and agent coordination is performed on top of this discrete abstraction.



(a) Visibility graph

(b) Voronoi diagram

(c) Cell decomposition

Figure 2.2: Classical approaches for constructing graph-based representations [14].

2.3 Search and Planning Foundations

Many MAPF approaches rely on classical graph search techniques for computing individual agent paths and for addressing coordination constraints. Accordingly, this section reviews optimal graph search and planning foundations that underlie a wide range of MAPF methods and will be revisited when discussing multi-agent coordination strategies.

2.3.1 Search Problems on Graphs

A search problem on a graph consists of finding a sequence of transitions that leads from a given initial state to a goal state while satisfying feasibility constraints and optimizing a specified cost criterion [16].

Formally, a search problem is defined by a state space, an initial state, a set of goal states, a transition model that specifies the allowable moves between states, and a cost function that assigns a non-negative cost to each transition or path [16]. When the state space is represented explicitly as a graph, vertices correspond to states and edges encode feasible transitions, possibly associated with traversal costs.

A solution to a graph search problem is a path in the graph that connects the initial vertex to a goal vertex. Among all feasible solutions, particular interest is often placed on optimal solutions, defined as those that minimize the cumulative cost along the path. The shortest path problem in weighted graphs is a canonical instance of this formulation and has been extensively studied in both theoretical and applied contexts [17].

Search algorithms typically operate by incrementally exploring the graph starting from the initial state, expanding states according to a specific strategy and maintaining a frontier of candidate paths. Different search strategies define different orders in which states are expanded, which directly affects properties such as completeness, optimality, and computational efficiency [18].

2.3.1.1 Dijkstra's Algorithm

Dijkstra's algorithm is a classical graph search algorithm for solving the shortest path problem in weighted graphs with non-negative edge costs. Originally introduced by Dijkstra [19], it provides a systematic procedure for computing the minimum-cost path from a given initial vertex to all other reachable vertices in the graph.

The algorithm operates by incrementally expanding vertices in order of increasing accumulated path cost from the initial state. At each step, the vertex with the lowest known cost is selected for expansion, and its outgoing edges are relaxed to update the cost of reaching neighboring vertices. This process continues until the goal vertex is reached or all reachable vertices have been explored [17]. A key property of Dijkstra's algorithm is that, under the assumption of non-negative edge costs, once a vertex is expanded, the cost associated with it is guaranteed to be optimal. As a result, the algorithm is both complete and optimal for this class of graphs [17].

From a search perspective, Dijkstra's algorithm can be viewed as a best-first search strategy in which the expansion order is determined by the accumulated path cost from the initial vertex. This accumulated cost, commonly denoted by $g(n)$ for a vertex n, corresponds to the sum of the costs of the edges along the path from the initial vertex to n,

$$g(n) = \sum_{(u,v)\in P_n} c(u,v) \tag{2.2}$$

where P_n denotes the path to n and $c(u,v)$ is the cost of edge (u,v).

In MAPF settings, it is often used as a baseline within more complex coordination frameworks, serving as a reference point for evaluating the benefits more advanced search strategies.

2.3.1.2 A* Search Algorithm

A* is a heuristic-based graph search algorithm that extends Dijkstra's algorithm by incorporating problem-specific information to guide the search toward the goal more efficiently. Originally introduced by Hart, Nilsson, and Raphael [18], A* is designed to compute optimal paths while reducing the number of explored states when compared to uninformed search methods.

The algorithm evaluates each vertex using an objective function

$$f(n) = g(n) + h(n) \tag{2.3}$$

where $g(n)$ denotes the accumulated cost from the initial vertex to vertex n, as defined in Equation (2.2), and $h(n)$ is a heuristic estimate of the remaining cost from n to a goal vertex.

This algorithm maintains a frontier of candidate vertices ordered by increasing values of $f(n)$ and iteratively expands the vertex with the lowest estimated total cost. When the heuristic function is admissible, meaning that it never overestimates the true remaining cost to the goal, A* is guaranteed to be both complete and optimal [18], [20].

(a) Dijkstra (b) A*

Figure 2.3: Comparison of node expansion patterns produced by Dijkstra's algorithm and A*.

Figure 2.3 illustrates the resulting difference in exploration behavior between Dijkstra's algorithm and A*. The light blue marker denotes the initial state, the green marker denotes the goal state, and darker blue cells correspond to expanded nodes. The environment is modeled as a grid to facilitate visualization, and the Manhattan distance heuristic is used to guide the A* search. As can be observed, the heuristic effectively directs the search toward the goal, resulting in a focused expansion pattern, whereas Dijkstra's algorithm exhibits a nearly radial expansion that explores the state space more uniformly.

From a search perspective, Dijkstra's algorithm can be seen as a special case of A* in which the heuristic function is identically zero. Under this interpretation, A* generalizes Dijkstra by allowing heuristic guidance while preserving optimality guarantees under appropriate conditions [17].

In MAPF settings, A* is frequently employed as a low-level search procedure within higher-level coordination algorithms, as well as for computing heuristic estimates used to guide multi-agent search processes.

2.4 Foundations of Parallel and Concurrent Computing

The MAPF problem involves the simultaneous evolution of multiple agents within a shared environment. As the number of agents increases, both the computational effort required to compute individual paths and the complexity of coordinating their interactions grow rapidly. For this reason, many MAPF approaches rely on principles drawn from parallel and concurrent computing.

For this reason, concepts from parallel and concurrent computing are particularly relevant for addressing coordination challenges in MAPF.

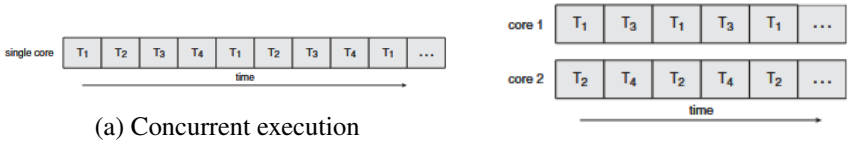
2.4.1 Parallelism and Concurrency

Parallelism and concurrency are closely related but conceptually distinct notions that describe different aspects of task execution within a computational system.

In this context, parallelism refers to the simultaneous execution of multiple computational tasks, typically with the objective of improving performance or reducing overall execution time. It

relies on the availability of multiple processing units, such as multi-core processors, and focuses on exploiting hardware resources to increase computational throughput [21].

Concurrency, in contrast, concerns the logical composition of multiple tasks that make progress overlapping periods of time, regardless of whether they are executed simultaneously in physical terms. These concepts are visually illustrated in Figure 2.4.



(a) Concurrent execution

(b) Parallel execution

Figure 2.4: Conceptual distinction between concurrency and parallelism [22].

2.4.2 Computational Models and Execution

Computational models provide an abstract description of how tasks are executed and how computational resources are organized during execution. These models are essential for reasoning about the structure, scalability, and coordination of computations involving multiple activities that progress over time.

2.4.2.1 Processes

A process represents an executing instance of a program together with its associated execution context. This context typically includes a private address space, program code, data, and operating system resources required for execution [23].

Processes provide strong isolation between executing programs, as each process operates within its own protected memory space. This isolation improves robustness and fault containment, since errors in one process cannot directly corrupt the state of another. However, inter-process communication generally incurs higher overhead, as it requires explicit mechanisms to exchange data across process boundaries [22].

From an execution perspective, processes constitute units of concurrency that provide strong isolation, at the cost of increased management and communication overhead.

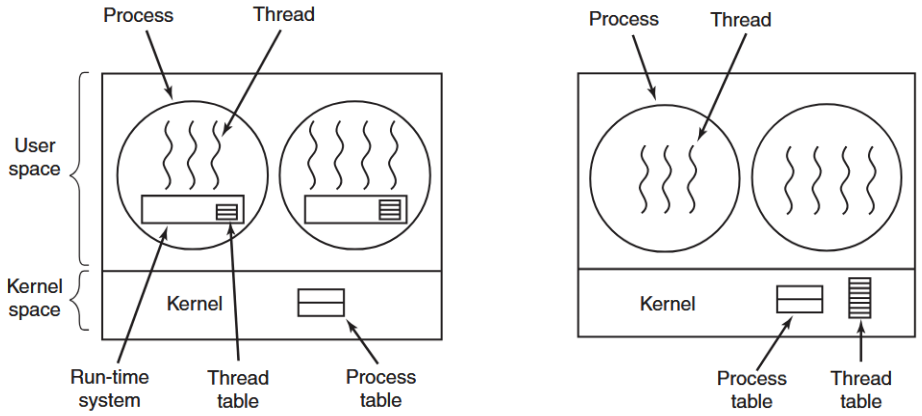
2.4.2.2 Threads

A thread represents a lower-level unit of execution within a process. Multiple threads within the same process share the process address space and resources, while maintaining independent execution contexts such as program counters and stacks [23]. This sharing enables efficient communication and cooperation between concurrent activities, as data can be accessed directly without inter-process communication mechanisms.

Threads can be classified according to how their management is implemented. In user-level threading models, thread creation, scheduling, and synchronization are handled by runtime systems

operating entirely in user space. As depicted in Fig. 2.5(a), the kernel is unaware of individual threads and schedules only processes, while thread tables and scheduling decisions are maintained by the runtime environment. This approach enables fast thread management but limits the ability to exploit multiple processors transparently [22].

In contrast, kernel-level threads, shown in Fig. 2.5(b), are managed directly by the operating system kernel. In this model, the kernel maintains thread tables and schedules threads independently, allowing threads belonging to the same process to be distributed across multiple processing units. While this enables true parallel execution on multi-core systems, it also introduces higher management overhead due to kernel involvement [21].



(a) User-level threading model (b) Kernel-level threading model

Figure 2.5: Conceptual comparison between user-level and kernel-level thread management. In user-level threading, thread scheduling and management are handled by a runtime system in user space, whereas in kernel-level threading these responsibilities are managed directly by the operating system kernel. Adapted from [23].

In both models, threads within a process typically share memory and other resources, adopting a shared-memory execution model. While this model supports efficient cooperation, it also requires careful coordination to ensure correct access to shared data. The mechanisms used to address these challenges are discussed in the following subsection.

2.4.3 Synchronization and Shared Resource Access

In concurrent execution models, multiple threads or tasks may access shared resources during their execution. While shared access enables cooperation and efficient communication, it also introduces challenges related to correctness, as uncontrolled interactions may lead to inconsistent or unintended system states.

2.4.3.1 Race Conditions

In concurrent execution models, incorrect synchronization may result in race conditions, where the behavior of a computation depends on the timing of concurrent executions. When multiple

threads access shared resources without proper coordination, the final system state may vary across different execution schedules, even when the same operations are performed [21].

Race conditions are particularly difficult to detect and reason about, as they often arise only under specific execution orders of concurrent tasks.

2.4.3.2 Critical Sections

To prevent race conditions, concurrent systems identify regions of execution that access shared resources as critical sections. A critical section denotes a portion of code in which a shared resource is accessed or modified and whose execution must be mutually exclusive.

Correct concurrent execution requires that critical sections operating on the same shared resource are not executed simultaneously by multiple threads, thereby ensuring consistency of the shared state [23].

2.4.3.3 Mutexes

Mutual exclusion mechanisms, commonly referred to as mutexes, provide a concrete means to enforce exclusive access to critical sections. A mutex ensures that at most one thread may enter a given critical section at any time.

By serializing access to shared resources, mutexes prevent concurrent modifications that could otherwise result in inconsistent system states. This behavior is illustrated in Fig. 2.6, where one process is blocked while another executes a critical section.

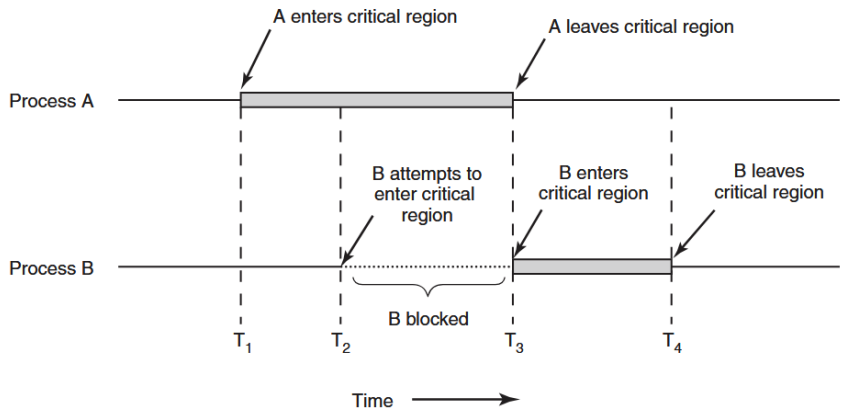


Figure 2.6: Illustration of mutual exclusion in a critical section. While Process A executes its critical section, Process B is blocked until the resource becomes available [23].

2.4.3.4 Semaphores

Semaphores provide a general synchronization for controlling access to shared resources and enforcing ordering constraints between concurrent tasks. Unlike mutexes, which are primarily intended to enforce exclusive access, semaphores support more flexible coordination patterns [24].

According to Arpaci-Dusseau [21], semaphores are commonly classified into two types:

- **Binary semaphores**, whose value is restricted to zero or one, and which can be used to enforce mutual exclusion in a manner similar to mutexes, though without ownership constraints;
- **Counting semaphores**, which take non-negative integer values and are used to manage access to a finite number of identical resources. The semaphore value represents the number of currently available resource instances and is decremented each time a resource is allocated to a process or thread.

2.4.3.5 Deadlocks

Another fundamental challenge in concurrent systems is deadlock, a state in which a set of threads becomes permanently blocked because each thread is waiting for resources held by others. Deadlocks arise from circular waiting dependencies and result in a loss of system progress, requiring careful design and coordination to avoid [22].

2.5 Summary

This chapter introduced the formal foundations of Multi-Agent Path Finding, including problem definition, environment representation, and classical graph search methods. In addition, fundamental concepts from parallel and concurrent computing were presented to motivate scalable execution and coordination strategies. These elements together provide the theoretical basis for the MAPF algorithms analyzed and proposed in the remainder of this dissertation.

Chapter 3

State of the Art

This chapter reviews the state of the art in MAPF, focusing on the main coordination paradigms and algorithmic approaches proposed in the literature. Section 3.1 surveys decentralized, centralized, hybrid, and learning-based coordination strategies, highlighting their respective strengths and limitations in multi-robot systems. Section 3.2 examines agent prioritization heuristics for sequential MAPF planners, with particular attention to their impact on solution feasibility and performance. Finally, Section 3.3 presents the CRIIS fleet coordination system, which operationalizes centralized MAPF planning and serves as the base for the methods investigated in this work.

3.1 Algorithmic Approaches to MAPF

The MAPF problem has been addressed through different coordination paradigms, reflecting distinct assumptions about system architecture and the distribution of coordination responsibilities. In multi-robot systems and fleet coordination, these paradigms exhibit characteristic trade-offs in terms of coordination quality, computational complexity, and robustness.

This chapter considers the following coordination paradigms:

- **Decentralized coordination**;
- **Centralized coordination**;
- **Hybrid coordination**;
- **Learning-based coordination**.

The following subsections discuss each of these paradigms in turn, introducing their main coordination principles and presenting representative algorithmic approaches to illustrate how these paradigms are realized in practice, with particular emphasis on centralized approaches, which constitute the algorithmic focus of this document.

3.1.1 Decentralized Coordination

Decentralized approaches distribute planning and decision-making among the agents themselves, without relying on a global coordination authority. In this paradigm, agents typically operate based on local information and, when available, limited communication with neighboring agents. Coordination emerges through local interactions, reactive behaviors, or distributed decision-making mechanisms, as illustrated in Figure 3.1.

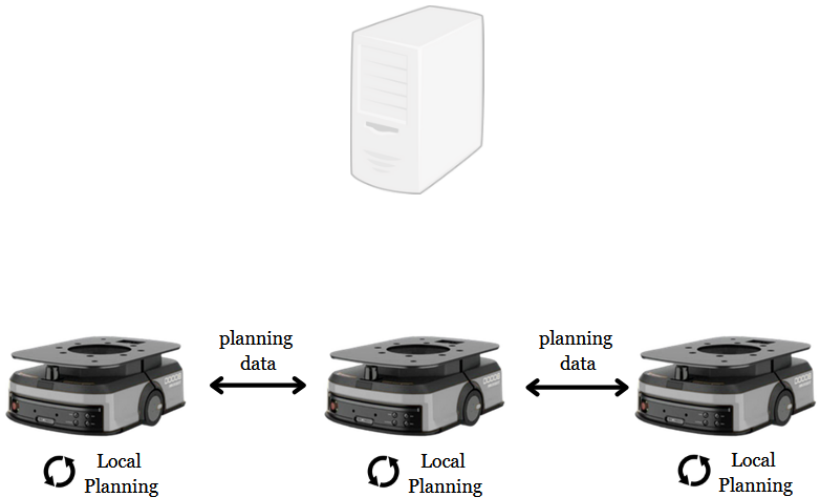


Figure 3.1: Illustration of a decentralized coordination architecture, where agents communicate directly with each other and no central planning entity is responsible for computing globally consistent paths.

Such approaches are particularly attractive in large-scale systems or in scenarios where communication is unreliable, as they avoid centralized computational bottlenecks and single points of failure [25]. Decentralized coordination has been extensively studied in the context of autonomous mobile robots, where agents must operate under partial observability and limited global knowledge [26].

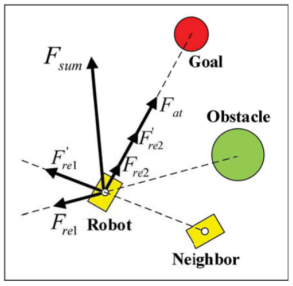
However, the absence of a global planning entity makes it difficult to ensure globally consistent behavior. Decentralized MAPF approaches generally provide limited guarantees with respect to completeness, deadlock avoidance, or solution quality, especially in environments with narrow passages or strong agent interactions [27]. As a result, while decentralized paradigms offer scalability, they often struggle to meet the predictability and safety requirements of tightly coordinated robotic fleets.

Several algorithmic approaches exemplify decentralized coordination in multi-robot systems. Reactive collision avoidance methods rely exclusively on local sensing and agent–agent interactions to prevent collisions, enabling scalable coordination without global planning, although with limited guarantees [28]. Distributed planning and negotiation-based approaches further extend this paradigm through peer-to-peer information exchange and local conflict resolution, improving robustness while avoiding centralized control [27].

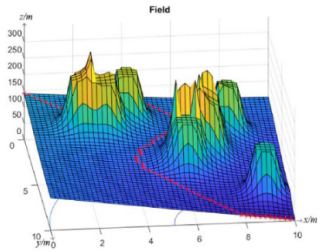
A representative example of decentralized coordination is provided by Artificial Potential Field (APF) methods. Originally proposed for single-agent navigation, APF approaches were designed to guide an agent towards a goal while avoiding obstacles by defining attractive and repulsive potential functions over the workspace [29]. More recent extensions have adapted APF-based methods to multi-robot settings by incorporating interaction forces between agents, which are treated as dynamic obstacles or sources of repulsive potentials. In such approaches, coordination emerges from local interactions, as each agent computes its motion based solely on locally available information. An example of this paradigm is the APF-CPP approach proposed by Wang et al. [30], where artificial potential fields are used for online multi-robot coverage path planning in a fully decentralized manner.

The resulting motion emerges from the combination of attractive forces towards goals and repulsive forces arising from obstacles and neighboring agents, allowing agents to react continuously to their local environment without requiring global coordination, as illustrated in Figure 3.2a.

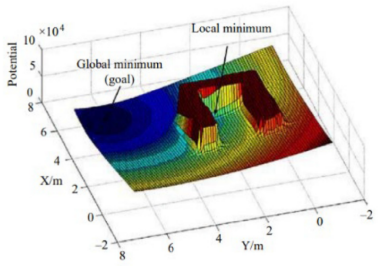
Although Figures 3.2b and 3.2c do not explicitly represent a multi-agent setting, they are highly illustrative of the fundamental behavior of potential fields and the emergence of local minima. Figure 3.2b depicts an idealized potential field, however, the interaction between attractive and repulsive potentials often gives rise to undesired local minima, as shown in Figure 3.2c. These local minima constitute a well-known limitation of APF-based methods, as agents may become trapped despite the existence of a feasible path to the goal [31].



(a) Attractive and repulsive forces acting on an agent in a multi-robot setting [30].



(b) Potential field [31].



(c) Potential field exhibiting local minima induced by obstacles [31].

Figure 3.2: Illustration of APF concepts.

Overall, the APF-based approach discussed above illustrates well several characteristic limitations of decentralized coordination methods. In particular, the reliance on local and reactive decision-making can lead to undesired behaviors such as local minima and deadlocks, reflecting the difficulty of achieving globally consistent solutions. As a result decentralized approaches typically provide limited guarantees regarding global completeness or solution quality.

3.1.2 Centralized Coordination

Centralized MAPF approaches rely on a single planning entity with access to the complete global state of the system, including the environment representation and the start and goal locations of all agents. This entity is responsible for computing coordinated, collision-free trajectories that ensure globally consistent behavior across the fleet. A typical centralized coordination architecture is depicted in Figure 3.3.

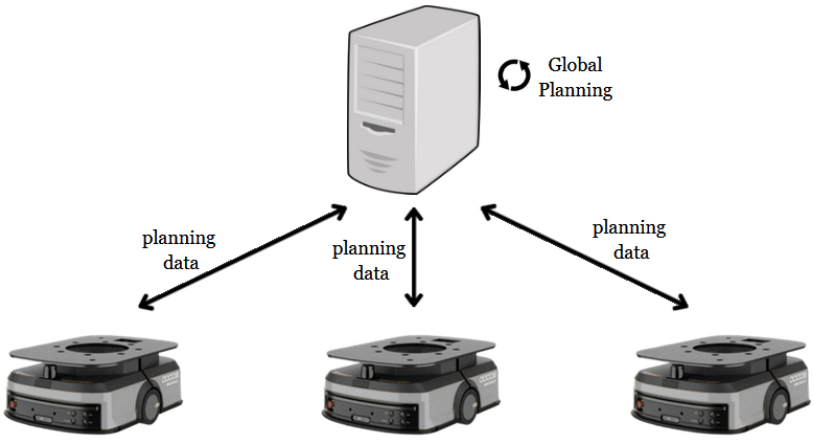


Figure 3.3: Illustration of a centralized coordination architecture, in which a central planning entity computes coordinated, collision-free trajectories for all agents based on complete global information.

Centralized coordination has long been adopted in industrial and structured environments, where reliable communication and centralized supervision are available [32]. This paradigm enables strong guarantees regarding safety, predictability, and coordination quality, and aligns well with fleet management systems that incorporate execution monitoring and replanning capabilities.

From a computational perspective, centralized MAPF approaches face inherent scalability challenges as the number of agents increases. As discussed in the background, the MAPF problem is NP-hard under standard formulations, which implies that planning rapidly becomes computationally expensive in densely populated or highly constrained environments. Consequently, practical centralized planners must carefully balance solution quality with computational efficiency. Nevertheless, access to complete global information allows centralized methods to reason explicitly about agent interactions and to resolve conflicts in a systematic and predictable manner, which remains a key advantage in scenarios requiring strong coordination guarantees.

Several algorithmic approaches have been proposed for centralized MAPF planning, differing in how conflicts are represented, detected, and resolved. Throughout this subsection, the discussion focuses on two representative methods, TEA* [33] and Conflict-Based Search (CBS) [3], which exemplify distinct strategies for centralized coordination.

3.1.2.1 TEA*

TEA* is a centralized multi-robot coordination approach that builds directly upon the classical A* search algorithm described in the background by extending the search space with an explicit temporal dimension. Instead of reasoning solely over spatial states, TEA* models robot motion in a space–time representation, where each state is augmented with a discrete time index, enabling the planner to reason explicitly about future agent positions and to avoid conflicts [33].

In this formulation, time progresses in discrete steps that correspond to the traversal of graph edges. Each feasible motion between two adjacent spatial nodes induces a transition to the next temporal layer, and the search is propagated forward up to a predefined temporal horizon $k_{\max}$. This layered space–time graph, illustrated in Figure 3.4, allows TEA* to encode both spatial feasibility and temporal ordering directly within the search process. By reserving nodes and edges across successive time layers during planning, the algorithm ensures that agents do not occupy the same resource at the same time, preventing future conflicts a priori rather than detecting and resolving them after paths have been computed.

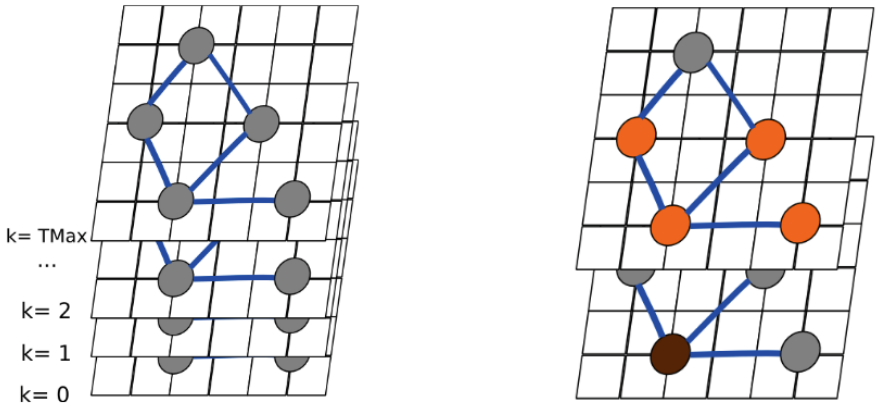


Figure 3.4: Illustration of the TEA* space–time representation [34].

In TEA*, paths are planned sequentially for each agent over a shared environment representation, which is commonly modeled as a graph. Once a path is computed for a given agent, the corresponding space–time trajectory is reserved and effectively treated as a dynamic obstacle for subsequently planned agents. During planning, nodes and edges occupied at specific time steps are marked as unavailable, preventing later agents from generating paths that would conflict with already planned motions. This temporal reservation mechanism allows TEA* to generate collision-free solutions, without requiring an explicit conflict detection and resolution stage [34].

A key characteristic of TEA* is its suitability for online execution and replanning. By maintaining a time-indexed representation of resource usage, the planner can update or recompute paths in response to execution delays, dynamic disturbances, or unexpected agent behavior. This capability makes TEA* particularly attractive for industrial and warehouse environments, where strict coordination requirements coexist with execution uncertainty and frequent replanning needs [33].

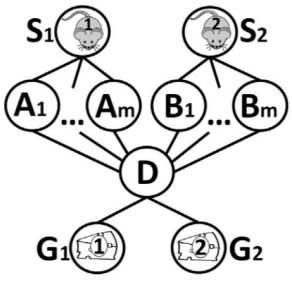
Several extensions of the original TEA* formulation have been proposed to address limitations arising from simplifying assumptions in the base model. In particular, Moura et al. [6] introduce a temporal optimization that accounts for robot kinematics, such as orientation changes and rotation costs, reducing the mismatch between the discrete planning model and real-world execution for differential-drive robots. Other works address livelock and deadlock phenomena by dynamically adjusting planning priorities or reordering the sequence in which agents are replanned [34].

Overall, the sequential nature of TEA* implies that the resulting solution may depend on the order in which agents are planned, potentially affecting solution quality or feasibility in densely constrained environments. This limitation, however, can be mitigated through the adoption of appropriate planning heuristics and agent ordering strategies. While this design choice sacrifices optimality guarantees under standard MAPF cost formulations, it offers a favorable trade-off in terms of computational efficiency.

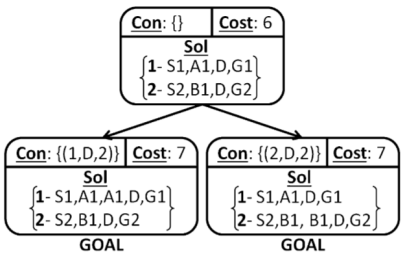
3.1.2.2 CBS

CBS is a centralized MAPF algorithm that resolves coordination through a hierarchical planning framework composed of a high-level conflict resolution process and a low-level path planner [3]. The key idea underlying CBS is to decouple individual path planning from conflict management, allowing agents to plan largely independently while explicitly resolving interactions when conflicts arise.

Figure 3.5a illustrates a simple MAPF instance in which two agents navigate a shared environment with alternative routes between start and goal locations. Although individual shortest paths may exist for each agent, conflicts can arise when agents attempt to occupy the same location or traverse the same edge at the same time, motivating the need for explicit coordination.



(a) Example MAPF instance illustrating potential conflicts between agents navigating a shared environment.



(b) Conflict Tree expansion in CBS, where conflicts are resolved by branching over alternative constraints.

Figure 3.5: Overview of CBS. (a) A MAPF instance in which independently planned paths may lead to conflicts. (b) High-level Conflict Tree used by CBS to systematically resolve conflicts through constraint generation. Adapted from [3].

The high-level search maintains a Conflict Tree (CT), illustrated in Figure 3.5b, where each node corresponds to a set of constraints and associated individual paths. When a conflict is detected, the corresponding CT node is expanded by generating alternative constraint sets, each prohibiting

one of the conflicting agents from performing the conflicting action. While this mechanism guarantees completeness and optimality under standard cost formulations, the number of conflicts and CT nodes may grow exponentially in the worst case, motivating several extensions to improve scalability.

At the low level, each agent computes a shortest path subject to a set of constraints. Recent extensions of CBS adopt Safe Interval Path Planning (SIPP) as the low-level planner in order to handle continuous time. This combination, commonly referred to as Continuous-Time CBS (CCBS), integrates SIPP into the CBS framework to enable optimal planning without discretizing time [35], [36]. SIPP operates by representing, for each spatial location, intervals of time during which the location is safe to occupy, allowing waiting actions and motion durations to be handled efficiently under temporal constraints. This integration makes CBS particularly well suited for domains where temporal feasibility plays a central role [36].

Several other variants of CBS have been proposed to address these limitations:

- **ECBS:** Bounded-suboptimal CBS introduces focal search to trade optimality for efficiency, allowing solutions within a predefined suboptimality bound [37].
- **CBSH:** Heuristic-guided CBS augments the high-level search with admissible heuristics that estimate the cost of unresolved conflicts, significantly reducing the size of the conflict tree while preserving optimality [38].
- **CBSH-RTC:** CBS variant with symmetry reasoning mitigate the combinatorial explosion of collision resolutions by identifying and pruning pairwise symmetric path combinations, substantially improving scalability and search efficiency. [39].

Overall, CBS and its variants exemplify a conflict-driven coordination paradigm, in which interactions between agents are explicitly detected and resolved through constraint generation at the high level.

3.1.2.3 Contrasting Centralized Coordination Strategies

Although both TEA* and CBS are centralized approaches to multi-agent path finding, they embody fundamentally different coordination strategies. CBS follows a conflict-resolution paradigm, in which agents initially plan independently and conflicts are detected and resolved through constraint generation at a higher level. As a result, coordination emerges from the iterative refinement of individual plans in response to detected interactions.

In contrast, TEA* adopts a conflict-avoidance strategy by reserving spatial resources across discrete time layers, TEA* prevents conflicts by construction, rather than detecting and resolving them after paths have been generated. These contrasting design choices lead to different trade-offs in terms of computational complexity, solution optimality, and suitability for online execution, motivating their joint analysis in applied fleet management scenarios.

3.1.3 Hybrid Coordination

Hybrid MAPF approaches seek to combine elements of decentralized and centralized coordination in order to leverage the advantages of both paradigms. Typically, these approaches adopt a hierarchical structure, in which high-level coordination decisions are made centrally, while lower-level planning or execution decisions are delegated to individual agents, as illustrated in Figure 3.6.

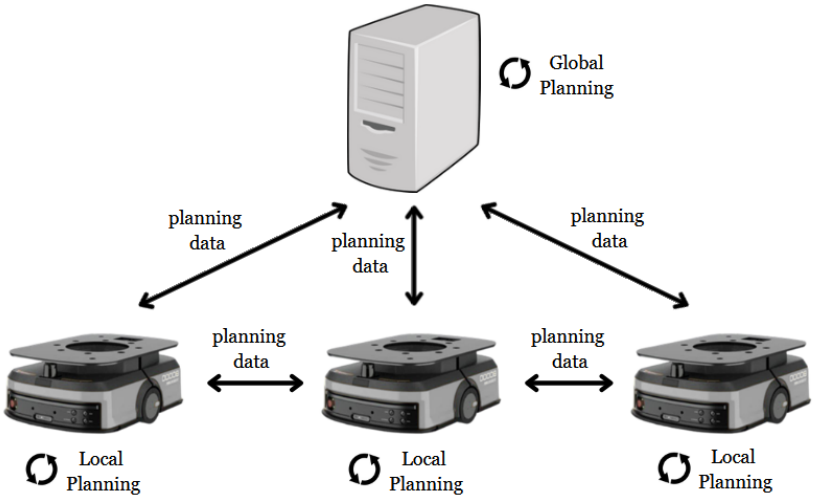


Figure 3.6: Illustration of a hybrid coordination architecture combining centralized decision-making at higher levels with decentralized planning or execution at the agent level.

The motivation for hybrid coordination lies primarily in scalability. By abstracting parts of the problem or limiting centralized planning to higher-level decisions, hybrid approaches aim to reduce the computational burden associated with fully centralized coordination, while still preserving some degree of global consistency [40].

Despite their potential benefits, hybrid and hierarchical approaches introduce additional complexity and may compromise theoretical guarantees. In particular, the separation between planning layers can lead to suboptimal solutions or coordination failures in complex or highly constrained environments. As such, hybrid paradigms are often tailored to specific application domains and system architectures.

Several representative approaches illustrate this hybrid coordination paradigm. Hierarchical MAPF methods employ centralized planning at a coarse level, such as region or waypoint assignment, while delegating fine-grained path planning to individual agents [41]. Other hybrid strategies rely on centralized global guidance combined with decentralized local execution, allowing agents to react autonomously to local disturbances while respecting global coordination constraints [40]. Region-based coordination approaches further reduce complexity by enforcing centralized constraints at the level of areas or traffic corridors, while allowing agents to plan locally within these regions [42], [43].

3.1.4 Learning-Based Coordination

More recently, learning-based approaches have been explored as an alternative paradigm for multi-agent coordination and path planning. These methods typically rely on machine learning techniques, such as reinforcement learning or imitation learning, to learn coordination policies from data rather than explicitly computing plans through search or optimization.

Learning-based coordination is motivated by the potential to generalize across environments and to adapt to complex, dynamic scenarios. In multi-agent settings, learning-based methods have been applied to collision avoidance, cooperative navigation, and decentralized decision-making [44], [45].

Representative examples of learning-based coordination include decentralized multi-agent reinforcement learning approaches, where agents learn local navigation and collision avoidance policies through interaction with the environment and other agents, without relying on explicit global planning [46]. Such methods emphasize scalability and adaptability, but typically lack formal guarantees.

Another prominent line of work combines classical MAPF planners with learning-based components. In these hybrid approaches, learning is used to guide heuristic selection, conflict resolution, or priority assignment within search-based planners, improving efficiency while preserving the underlying planning structure [44].

Despite promising results, learning-based approaches face significant challenges in the context of MAPF. These include the need for large amounts of training data and the difficulty of providing strong guarantees regarding safety, completeness, or optimality. As a result, learning-based coordination is often considered complementary to classical MAPF methods rather than a direct replacement, particularly in safety-critical fleet management applications.

3.2 Agent Prioritization Heuristics

Sequential and prioritized planning approaches constitute an important class of MAPF solvers, particularly in centralized settings where agents are planned one after another according to a predefined priority ordering. As discussed in the previous section, TEA* follows this paradigm by planning agents sequentially in a shared space–time representation. A well-known characteristic of such approaches is their strong dependence on the chosen agent ordering, which can significantly influence solution feasibility, solution quality, and computational performance. This sensitivity has motivated extensive research on agent prioritization heuristics, aiming to determine planning orders that reduce conflicts, deadlocks, or excessive delays. However, as consistently highlighted in the literature and recent surveys, no single heuristic dominates across all environments and task configurations, with performance being highly scenario-dependent [5], [47]. This section reviews representative classes of agent prioritization heuristics proposed in the context of prioritized MAPF planning.

3.2.1 Randomized and Multi-Ordering Strategies

The simplest approach to mitigate the sensitivity of sequential MAPF planners to agent ordering is randomized prioritization, which is commonly used as a baseline strategy. In this approach, agents are assigned priorities according to random permutations, and planning is repeated multiple times. Despite its simplicity, randomized ordering can occasionally outperform carefully designed heuristics by avoiding systematic bias toward unfavorable priority configurations [47]. As such, it is frequently adopted as a robustness baseline in empirical evaluations.

3.2.2 Path-Length-Based Heuristics

One of the most widely used classes of prioritization heuristics is based on estimated path length. The underlying intuition is that agents with longer or more constrained paths should be planned earlier, as they typically have fewer alternative routing options.

A common variant is the *Longest Path First* heuristic, where agents are ordered in descending order of their shortest-path distance to the goal [5]. Conversely, *Shortest Path First* heuristics prioritize agents with shorter paths, favoring early completion but often increasing the likelihood of conflicts for agents with longer routes. While path-length-based heuristics are simple to compute and effective in sparse environments, they do not explicitly account for interactions between agents and may perform poorly in congested or highly coupled scenarios.

3.2.3 Conflict-Aware Prioritization

To address the limitations of purely distance-based heuristics, several approaches incorporate conflict awareness into the prioritization process. These heuristics estimate the likelihood of an agent being involved in conflicts based on spatial overlap with other agents' paths or shared critical regions.

Early approaches prioritize agents whose shortest paths intersect frequently with others, under the assumption that planning them first reduces downstream conflicts [47]. While conflict-aware heuristics often improve success rates in dense environments, they introduce additional computational overhead and typically rely on approximate conflict estimation.

3.2.4 Environment-Aware and Bottleneck-Based Heuristics

Another class of prioritization heuristics exploits structural properties of the environment. Agents whose paths traverse narrow corridors, bottlenecks, or shared resources are prioritized earlier, as these regions are particularly prone to congestion and deadlocks.

Such heuristics are especially relevant in structured environments such as warehouses or factories, where spatial constraints strongly influence coordination difficulty [47]. Environment-aware prioritization has been shown to reduce deadlocks and improve execution predictability, particularly when combined with time-augmented planning approaches.

3.3 Fleet Coordination System

This section describes the architecture of the CRIIS fleet coordination system, which is designed as a centralized coordination framework built around TEA*, complemented with supervision and graph-processing mechanisms to handle real-world scenarios [4]. The system is implemented on top of the Robot Operating System (ROS), which provides the communication middleware and execution infrastructure supporting both centralized coordination and distributed robot control.

The architecture follows a modular design that separates global coordination responsibilities from local robot control. As illustrated in Figure 3.7, at a high level, the system is composed of two main components. The *Central Coordination Module* is responsible for global planning, supervision, and coordination, maintaining a global view of the environment and the fleet state. In contrast, the *Agent Control Module*, instantiated on each robot, is responsible for executing the received plans, interfacing with the robot's navigation stack, and providing execution feedback to the central supervisor [4].

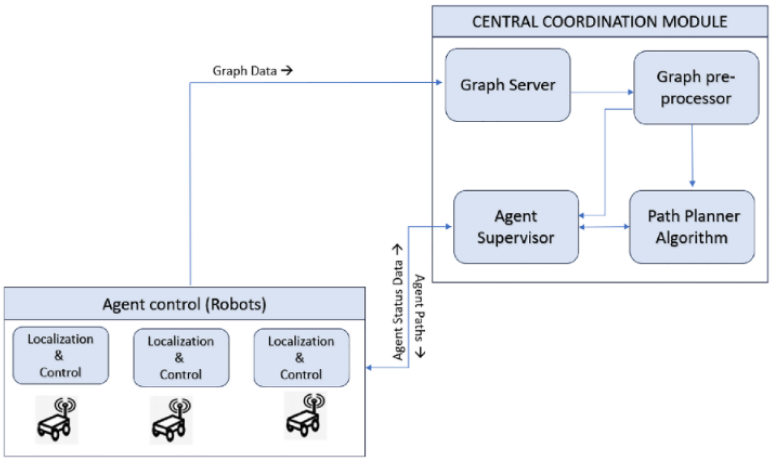


Figure 3.7: High-level architecture of the CRIIS fleet coordination system [4].

The remainder of this section focuses on a detailed description of the Central Coordination Module and its internal components.

3.3.1 Central Coordination Module

The Central Coordination Module is responsible for global decision-making and coordination across the robotic fleet. It maintains a centralized representation of the environment and agent states, computes coordinated plans, and supervises execution[4].

As summarized in Figure 3.8, this module comprises four core components: the Graph Server, the Graph Pre-Processor, the Path Planner, and the Supervisor. Interaction with the robots' navigation systems is mediated through a Navigation Handler, which provides an abstraction layer for plan dispatch and execution feedback.

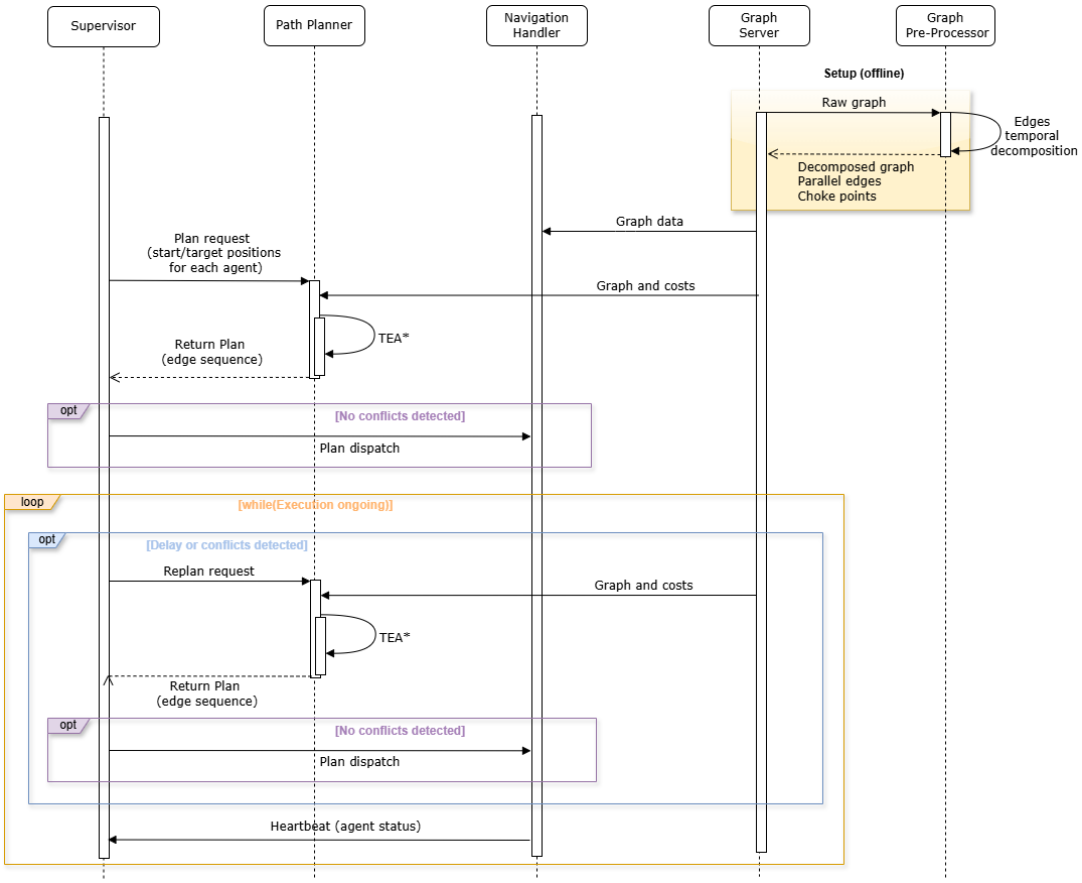


Figure 3.8: Sequence diagram of the CRIIS fleet coordination workflow.

3.3.1.1 Graph Server

The Graph Server is responsible for storing and providing access to the global navigation graph shared by all agents in the fleet. This graph encodes the topological structure of the environment, including nodes, directed edges, and their associated traversal costs. In the CRIIS fleet coordination system, these costs are expressed in temporal terms, enabling time-aware planning and synchronization among agents [4].

Additionally, the Graph Server exposes a query interface that allows other components, namely the Path Planner, to retrieve graph topology, edge attributes, and metadata required for coordination. By decoupling graph management from planning logic, the system promotes modularity and facilitates future extensions, such as dynamic cost updates or graph augmentation strategies.

3.3.1.2 Graph Pre-Processor

The Graph Pre-Processor operates during an offline configuration phase and augments the raw navigation graph with information required for multi-robot coordination, transforming a purely geometric representation into a coordination-aware graph [4]. Its responsibilities include identifying footprint-related constraints, such as parallel edges that cannot be safely occupied simultaneously,

and detecting critical shared resources (choke points) like narrow corridors or doorways that are prone to congestion and deadlocks.

Additionally, since TEA* advances time in discrete steps associated with edge traversals, the pre-processor performs edge discretization and normalization according to a global temporal resolution. This ensures consistent traversal times across the graph, improving temporal alignment between agents and enabling more accurate space–time reasoning during planning, thereby reducing synchronization errors and unnecessary replanning [4].

3.3.1.3 Path Planner

The Path Planner is the core decision-making component responsible for computing coordinated, collision-free trajectories for the entire fleet. In the CRIIS system, this functionality is implemented using the TEA* algorithm [4].

Planning requests include the current symbolic positions and target goals of all agents. Upon receiving a request, the Path Planner retrieves the navigation graph and associated traversal costs from the Graph Server and computes time-consistent sequences of edges for each agent. By reasoning over a shared space–time representation, the planner can explicitly account for interactions between agents, reserving graph resources over time and preventing conflicts through temporal exclusion mechanisms.

The centralized nature of the Path Planner enables globally coherent solutions, particularly in environments with shared resources or high robot density. This approach allows conflicts to be resolved at planning time rather than during execution, reducing the likelihood of deadlocks, emergency stops, or excessive local replanning [4].

3.3.1.4 Navigation Handler

The Navigation Handler serves as the integration layer between the centralized coordination logic and the robots' low-level navigation systems. It exposes an API that supports plan dispatch, translation of symbolic graph-based edge sequences into commands compatible with the robot navigation stack, and continuous monitoring of execution progress.

During execution, the Navigation Handler collects feedback such as the current symbolic state, edge completion status, and execution delays, and reports this information to the Supervisor. By isolating execution-specific concerns from planning logic, it enhances system modularity and allows the coordination framework to remain agnostic to the underlying robot platforms and navigation implementations [4].

3.3.1.5 Supervisor

The Supervisor is responsible for orchestrating the overall coordination loop by closing the feedback cycle between planning and execution. It aggregates execution feedback provided by the Navigation Handler, monitors the progress of each agent, and maintains a global view of the system state [4].

Based on this information, the Supervisor detects abnormal or unexpected conditions, such as execution delays, deviations between planned and executed trajectories, or emerging conflicts caused by dynamic disturbances. When such situations are identified, the Supervisor triggers replanning requests to the Path Planner.

This supervision mechanism is a key contributor to system robustness under real-world uncertainty, including variable execution times, communication latency, and dynamic obstacles. By enabling adaptive replanning while preserving centralized coordination, the Supervisor ensures that the fleet can maintain safe and efficient operation even when assumptions made at planning time are violated [4].

3.4 Summary

This chapter reviewed the main coordination paradigms and algorithmic approaches to MAPF, with particular emphasis on centralized planning methods. While decentralized and learning-based approaches offer scalability and adaptability, their limited guarantees regarding safety, predictability, and solution quality restrict their applicability in tightly coordinated fleet management scenarios.

Motivated by these constraints, the CRIIS fleet coordination system adopts a centralized, time-aware coordination architecture that explicitly reasons about agent interactions and execution timing. By integrating TEA* within a broader framework that includes graph preprocessing, supervision, and execution monitoring, CRIIS bridges the gap between theoretical MAPF formulations and the practical requirements of real-world robotic fleets, such as execution uncertainty, communication delays, and physical interaction constraints.

A central design choice in CRIIS is the use of sequential, prioritized planning, which enables responsive and scalable coordination but introduces a strong dependence on agent ordering. Although numerous prioritization heuristics have been proposed in the literature, recent surveys indicate that agent prioritization remains comparatively underexplored despite its practical relevance. This observation directly motivates the investigation of alternative prioritization strategies and multi-ordering approaches pursued in this work.

Chapter 4

Baseline System Stabilization

This chapter presents the stabilization process applied to the baseline coordination pipeline before proceeding with the planner improvements. During preliminary real-time experiments involving the complete system architecture, several inconsistencies were identified in the way planning and replanning requests, as well as agent state updates, were processed by the Supervisor component.

To address these issues, synchronization and batching mechanisms were introduced to provide a more consistent and reliable coordination framework for the subsequent development stages.

4.1 Initial System Instability Issues

Before introducing new prioritization heuristics and parallel planning strategies, it was first necessary to stabilize the baseline coordination system. During preliminary experiments, a set of undesired behaviors was identified in the coordination pipeline involving the robots' Navigation Handlers, the Supervisor, and the Path Planner during planning and replanning operations.

One of the most critical issues consisted of robots unexpectedly stopping or deviating from the expected execution flow immediately after receiving newly computed plans. In several situations, the first segments of the replanned paths no longer corresponded to the robots' current physical positions, leading to large tracking errors at the controller level. As a consequence, robots could temporarily behave inconsistently with the assumptions made by the global coordination system, potentially compromising the safety and consistency of the execution.

A preliminary analysis suggested that these behaviors were strongly related to the way planning requests and agent state updates were processed by the Supervisor component. More specifically, replanning operations were frequently triggered using outdated or temporally inconsistent agent states. Since the planner may require a non-negligible amount of computation time, especially in large environments or high-density multi-robot scenarios, the discrepancy between the planner's internal representation of the system and the real robot states could further increase before the generated plan was finally applied.

These observations motivated a deeper analysis of the temporal behavior of the coordination pipeline, with particular focus on message propagation delays, ROS callback processing, and replanning triggering mechanisms.

4.2 Temporal Analysis of the Coordination System

To better understand the origin of the previously described inconsistencies, a temporal analysis of the coordination pipeline was conducted. Figure 4.1 illustrates a simplified representation of the most relevant events involved in the interaction between the robots, the Supervisor, and the planner.

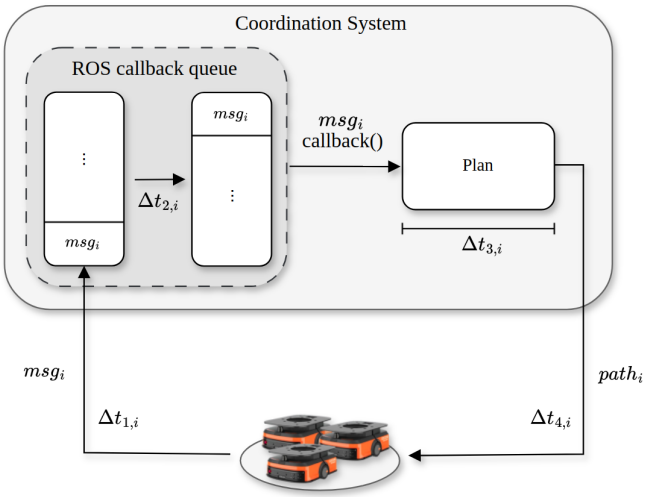


Figure 4.1: Illustration of a centralized coordination architecture, in which a central planning entity computes coordinated, collision-free trajectories for all agents based on complete global information.

The presented abstraction focuses primarily on the temporal intervals associated with message transmission, ROS callback queue processing, and planner execution. In particular:

- $\Delta t_{1,i}$ corresponds to the transmission delay between robot i and the Supervisor;
- $\Delta t_{2,i}$ represents the time spent in the ROS callback queue, including both waiting and callback processing time;
- $\Delta t_{3,i}$ denotes the planner execution time required to compute a new plan after a planning request is received;
- $\Delta t_{4,i}$ corresponds to the transmission delay associated with sending the newly computed plan back to robot i.

The analysis revealed that the dominant source of temporal inconsistency was not message transmission itself, but rather the accumulation of delays introduced by sequential callback processing and repeated planner executions.

In practice, while the planner was executing, newly received state updates and planning requests continued to accumulate in the callback queue. Consequently, subsequent planning operations were often initialized using system states that no longer accurately represented the real positions and progression of the robots at the time the new plans became available.

This issue became particularly relevant during scenarios involving multiple nearly simultaneous planning requests or high-frequency state updates, where the coordination system effectively operated on temporally inconsistent snapshots of the environment.

Chapter 5

Prioritization Heuristics Design and Development

Chapter 6

Parallelization and Heuristic Dispatching Framework

Chapter 7

Experimental Evaluation and Performance Analysis

Chapter 8

Conclusion

This document established the foundations and technical direction for addressing centralized multi-robot coordination under the MAPF formulation. Through the analysis of MAPF fundamentals and state-of-the-art coordination paradigms, it became clear that centralized approaches remain particularly well suited for scenarios requiring strong safety, predictability, and global consistency guarantees, despite their inherent scalability challenges.

The detailed study of centralized planning methods highlighted the advantages of TEA* in practical fleet coordination systems, namely its explicit handling of temporal constraints and its suitability for online replanning. However, the sequential and prioritized nature of TEA* was identified as a key limitation, as planning outcomes are highly sensitive to agent ordering and no single prioritization heuristic performs consistently well across different environments and task configurations.

Motivated by this observation, a preliminary solution was proposed that extends the existing TEA*-based planner through parallel exploration of multiple priority orderings. By executing several sequential TEA* instances concurrently and selecting the best resulting plan, the proposed approach aims to improve robustness and solution quality without altering the core architecture of the centralized coordination framework. The discussion of alternative execution and synchronization strategies further emphasized the potential to balance planning latency and exploration depth through tunable parameters.

Overall, the conclusions drawn in this document motivate the dissertation phase, which will focus on validating whether parallel prioritization can effectively mitigate ordering sensitivity in TEA*-based fleet coordination systems under realistic conditions.

References

[1] R. Stern, N. R. Sturtevant, A. Felner, S. Koenig, H. Ma, T. T. Walker, J. Li, D. Atzmon, L. Cohen, T. K. S. Kumar, E. Boyarski, and R. Barták, "Multi-agent pathfinding: Definitions, variants, and benchmarks," *Proceedings of the 12th International Symposium on Combinatorial Search, SoCS 2019*, pp. 151–158, Jun. 2019, ISSN: 2832-9171. DOI: 10.1609/socs.v10i1.18510.

[2] J. Yu and S. M. Lavalle, "Planning optimal paths for multiple robots on graphs," *Proceedings - IEEE International Conference on Robotics and Automation*, pp. 3612–3617, Apr. 2012, ISSN: 10504729. DOI: 10.1109/ICRA.2013.6631084.

[3] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, vol. 219, pp. 40–66, Feb. 2015, ISSN: 0004-3702. DOI: 10.1016/J.ARTINT.2014.11.006.

[4] D. M. Matos, P. Costa, H. Sobreira, A. Valente, and J. Lima, "Efficient multi-robot path planning in real environments: A centralized coordination system," *International Journal of Intelligent Robotics and Applications*, vol. 9, pp. 217–244, 1 Mar. 2025, ISSN: 2366598X. DOI: 10.1007/s41315-024-00378-3.

[5] D. Silver, "Cooperative pathfinding," *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 1, pp. 117–122, 1 Jun. 2005, ISSN: 2334-0924. DOI: 10.1609/AIIDE.V1I1.18726.

[6] P. Moura, P. Costa, J. Lima, and P. Costa, "A temporal optimization applied to time enhanced a," vol. 2116, American Institute of Physics Inc., Jul. 2019, ISBN: 9780735418547. DOI: 10.1063/1.5114225.

[7] P. Surynek, "An optimization variant of multi-robot path planning is intractable," vol. 24, AAAI Press, Jul. 2010, pp. 1261–1263, ISBN: 9781577354642. DOI: 10.1609/AAAI.V24I1.7767.

[8] T. Geft, "Fine-grained complexity analysis of multi-agent path finding on 2d grids," *The International Symposium on Combinatorial Search*, vol. 16, pp. 20–28, 1 May 2023, ISSN: 28329163. DOI: 10.1609/socs.v16i1.27279.

[9] J. Gao, Y. Li, X. Li, K. Yan, K. Lin, and X. Wu, "A review of graph-based multi-agent pathfinding solvers," *Knowledge-Based Systems*, vol. 283, Jan. 2024, ISSN: 09507051. DOI: 10.1016/J.KNOSYS.2023.111121.

[10] A. Yang, Q. Niu, W. Zhao, K. Li, and G. W. Irwin, "An efficient algorithm for grid-based robotic path planning based on priority sorting of direction vectors," vol. 6329 LNCS, Springer, Berlin, Heidelberg, 2010, pp. 456–466, ISBN: 978-3-642-15597-0. DOI: 10.1007/978-3-642-15597-0_50.

[11] H. Ma, "Graph-based multi-robot path finding and planning," *Current Robotics Reports*, vol. 3, pp. 77–84, 3 Jun. 2022. DOI: 10.1007/s43154-022-00083-8.

[12] T. Lozano-Pérez and M. A. Wesley, "An algorithm for planning collision-free paths among polyhedral obstacles," *Communications of the ACM*, vol. 22, pp. 560–570, 10 Oct. 1979, ISSN: 15577317. DOI: 10.1145/359156.359164.

[13] F. Aurenhammer, "Voronoi diagrams—a survey of a fundamental geometric data structure," *ACM Computing Surveys (CSUR)*, vol. 23, pp. 345–405, 3 Jan. 1991, ISSN: 15577341. DOI: 10.1145/116873.116880.

[14] J.-C. Latombe, *Robot Motion Planning*. Springer, 1991.

[15] K. L. Seth Hutchinson Howie Choset, *Principles of Robot Motion*. 2005, vol. 53, pp. 1079–1083, ISBN: 9780262033275. [Online]. Available: https://mitpress.mit.edu/9780262033275/principles-of-robot-motion/.

[16] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Prentice Hall, 2010, ISBN: 978-0136042594.

[17] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, Sep. 2009, ISBN: 9780262033848. [Online]. Available: https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/.

[18] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, pp. 100–107, 2 1968, ISSN: 21682887. DOI: 10.1109/TSSC.1968.300136.

[19] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1 Dec. 1959, ISSN: 0029599X. DOI: 10.1007/BF01386390.

[20] T. L. Rankin, *Heuristics: Intelligent Search Strategies for Computer Problem Solving. Judea Pearl. Addison-Wesley Publishing. 1984. 382 pp.* 1986, vol. 7, pp. 87–89. DOI: https://doi.org/10.1609/aimag.v7i1.534.

[21] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/.

[22] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018, ISBN: 978-1119456339.

[23] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2015, ISBN: 978-0133591620.

[24] E. W. Dijkstra, "Cooperating sequential processes," pp. 43–112, 1968.

[25] L. Siefke, V. Sommer, B. Wudka, C. Thomas, L. Siefke, V. Sommer, B. Wudka, and C. Thomas, "Robotic systems of systems based on a decentralized service-oriented architecture," *Robotics 2020, Vol. 9,*, vol. 9, pp. 1–10, 4 Sep. 2020, ISSN: 2218-6581. DOI: 10.3390/ROBOTICS9040078.

[26] P. Flocchini, G. Prencipe, N. Santoro, and P. Widmayer, "Distributed coordination of a set of autonomous mobile robots," *IEEE Intelligent Vehicles Symposium, Proceedings*, pp. 480–485, 2000. DOI: 10.1109/IVS.2000.898389.

[27] M. Čáp, P. Novák, J. Vokřínek, and M. Pěchouček, *Asynchronous Decentralized Algorithm for Space-Time Cooperative Pathfinding*. Oct. 2012. [Online]. Available: https://arxiv.org/abs/1210.6855v1.

[28] J. D. V. Berg, M. Lin, and D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation," *Proceedings - IEEE International Conference on Robotics and Automation*, pp. 1928–1935, 2008, ISSN: 10504729. DOI: 10.1109/ROBOT.2008.4543489.

[29] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The International Journal of Robotics Research*, vol. 5, pp. 90–98, 1 1986, ISSN: 02783649. DOI: 10.1177/027836498600500106.

[30] Z. Wang, X. Zhao, J. Zhang, N. Yang, P. Wang, J. Tang, J. Zhang, and L. Shi, "Apf-cpp: An artificial potential field based multi-robot online coverage path planning approach," *IEEE Robotics and Automation Letters*, vol. 9, pp. 9199–9206, 11 2024, ISSN: 23773766. DOI: 10.1109/LRA.2024.3432351.

[31] Z. Wu, J. Dai, B. Jiang, and H. R. Karimi, "Robot path planning based on artificial potential field with deterministic annealing," *ISA Transactions*, vol. 138, pp. 74–87, Jul. 2023, ISSN: 00190578. DOI: 10.1016/J.ISATRA.2023.02.018.

[32] P. Caloud, W. Choi, J. C. Latombe, C. L. Pape, and M. Yim, "Indoor automation with many mobile robots," *IEEE International Conference on Intelligent Robots and Systems*, vol. 1990-July, pp. 67–72, 1990, ISSN: 21530866. DOI: 10.1109/IROS.1990.262370.

[33] J. Santos, P. Costa, L. F. Rocha, A. P. Moreira, and G. Veiga, "Time enhanced a: Towards the development of a new approach for multi-robot coordination," *Proceedings of the IEEE International Conference on Industrial Technology*, vol. 2015-June, pp. 3314–3319, June Jun. 2015. DOI: 10.1109/ICIT.2015.7125589.

[34] J. Santos, P. Costa, L. Rocha, K. Vivaldini, A. P. Moreira, and G. Veiga, "Validation of a time based routing algorithm using a realistic automatic warehouse scenario," *Advances in Intelligent Systems and Computing*, vol. 418, pp. 81–92, 2016, ISSN: 2194-5365. DOI: 10.1007/978-3-319-27149-1_7.

[35] A. Andreychuk, K. Yakovlev, E. Boyarski, and R. Stern, "Improving continuous-time conflict based search," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 11 220–11 227, 13 May 2021, ISSN: 2374-3468. DOI: 10.1609/AAAI.V35I13.17338.

[36] M. Phillips and M. Likhachev, "Sipp: Safe interval path planning for dynamic environments," *Proceedings - IEEE International Conference on Robotics and Automation*, pp. 5628–5635, 2011, ISSN: 10504729. DOI: 10.1109/ICRA.2011.5980306.

[37] M. Barer, G. Sharon, R. Stern, and A. Felner, "Suboptimal variants of the conflict-based search algorithm for the multi-agent pathfinding problem," *Proceedings of the International Symposium on Combinatorial Search*, vol. 5, pp. 19–27, 1 2014, ISSN: 2832-9163. DOI: 10.1609/SOCS.V5I1.18315.

[38] A. Felner, J. Li, E. Boyarski, H. Ma, L. Cohen, T. K. Kumar, and S. Koenig, "Adding heuristics to conflict-based search for multi-agent path finding," *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 28, pp. 83–87, Jun. 2018, ISSN: 2334-0843. DOI: 10.1609/ICAPS.V28I1.13883.

[39] J. Li, D. Harabor, P. J. Stuckey, H. Ma, G. Gange, and S. Koenig, "Pairwise symmetry reasoning for multi-agent path finding search," *Artificial Intelligence*, vol. 301, p. 103 574, 2021, ISSN: 0004-3702. DOI: https://doi.org/10.1016/j.artint.2021.103574.

[40] B. Muppasani, R. Dey, B. Srivastava, and V. Narayanan, "Scalable multi-agent path finding using collision-aware dynamic alert mask and a hybrid execution strategy," Oct. 2025. [Online]. Available: https://arxiv.org/abs/2510.09469v1.

[41] T. Standley, "Finding optimal solutions to cooperative pathfinding problems," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 24, pp. 173–178, 1 Jul. 2010, ISSN: 2374-3468. DOI: 10.1609/AAAI.V24I1.7564.

[42] H. Zhang, M. Yao, Z. Liu, J. Li, L. Terr, S. H. Chan, T. K. S. Kumar, and S. Koenig, "A hierarchical approach to multi-agent path finding," *Proceedings of the International Symposium on Combinatorial Search*, vol. 12, pp. 209–211, 1 Jul. 2021, ISSN: 2832-9163. DOI: 10.1609/SOCS.V12I1.18586.

[43] C. Leet, J. Li, and S. Koenig, "Shard systems: Scalable, robust and persistent multi-agent path finding with performance guarantees," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, pp. 9386–9395, 9 Jun. 2022, ISSN: 2374-3468. DOI: 10.1609/AAAI.V36I9.21170.

[44] S. Wang, H. Xu, Y. Zhang, J. Lin, C. Lu, X. Wang, and W. Li, "Where paths collide: A comprehensive survey of classic and learning-based multi-agent pathfinding," May 2025. [Online]. Available: https://arxiv.org/abs/2505.19219v2.

[45] J. Hu, "A survey on reinforcement learning-based multi-agent path planning," *ITM Web of Conferences*, vol. 78, Jan. 2025. DOI: 10.1051/itmconf/20257801015.

[46] M. Everett, Y. F. Chen, and J. P. How, "Collision avoidance in pedestrian-rich environments with deep reinforcement learning," *IEEE Access*, vol. 9, pp. 10 357–10 377, 2021. DOI: 10.1109/ACCESS.2021.3050338.

[47] J. R. Heselden, G. P. Das, J. R. Heselden, and G. P. Das, "Heuristics and rescheduling in prioritised multi-robot path planning: A literature review," *Machines 2023, Vol. 11,*, vol. 11, 11 Nov. 2023, ISSN: 2075-1702. DOI: 10.3390/MACHINES11111033.